

119. Un switch, et une fonction par état

La boucle ne fait plus qu'aiguiller : chaque état a sa fonction, et une seule fonction change d'état.

1. Ce qu'il faut comprendre

Avec un vrai jeu dans l'état `JEU`, le `if ... else if` de la leçon d'avant grossirait jusqu'à ne plus tenir sur l'écran. On range donc **chaque état dans sa fonction** : `titre()`, `jeu()`, `perdu()`.

Ce qui est nouveau ici : le `switch` sur l'état, et `changer()`. Le `switch` (chapitre 2) aiguille vers la fonction de l'état en cours ; `main` ne fait plus rien d'autre. Et **une seule** fonction, `changer(nouvel)`, a le droit de changer d'état : elle efface l'écran, dessine le nouveau, et prépare ses variables.

Pourquoi une seule ? Parce que chaque état a quelque chose à faire **en entrant** : la partie remet le score à 0 et lance le chrono ; l'écran de fin affiche le score. Si trois endroits du programme changeaient `etat` à la main, on oublierait une fois sur deux de remettre le score à zéro.

`changer(Etat nouvel)` prend un argument de type `Etat` : on ne peut lui passer que TITRE, JEU ou PERDU. Même le compilateur lit le programme plus facilement.

La partie est un petit jeu : **appuyer sur A le plus de fois possible en cinq secondes**. Le front de A (un appui, un point), le compte à rebours du chapitre 11, et quand il tombe à 0 : `changer(PERDU)`.

Le changement d'écran se fait **écran éteint** : `ecran(0)`, on efface et on écrit, `ecran(1)`. Rien ne se voit à moitié dessiné.

À toi : ajoute l'état `PAUSE` avec sa fonction `pause()`, et vérifie que le temps ne s'écoule pas pendant la pause.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1288 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      68 o ← la console, à l'allumage — la mise en route de la console
appuiStart()   40 o ← titre(), perdu() — ta fonction
changer()      224 o ← titre(), jeu(), perdu(), main() — ta fonction
titre()        15 o ← main() — ta fonction
jeu()          97 o ← main() — ta fonction
perdu()        14 o ← main() — ta fonction
main()         55 o ← démarrage — ta fonction
```

VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← ecran() – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← texte() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
CopierTuiles	85 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	432 o	← CopierTuiles – 27 lettres sur 43
"	..."	20 o ← texte() – un texte gravé
"LE JEU DU A"	11 o	← texte() – un texte gravé
"START: JOUER"	12 o	← texte() – un texte gravé
"APPUIE SUR A!"	13 o	← texte() – un texte gravé
"POINTS"	6 o	← texte() – un texte gravé
"TEMPS"	5 o	← texte() – un texte gravé
"FINI!"	5 o	← texte() – un texte gravé
"POINTS"	6 o	← texte() – un texte gravé
"START: TITRE"	12 o	← texte() – un texte gravé

Lettres gravées : ADEFIJLMNOPRSTU0123456789!:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 9	105 o sur place, dans changer() + AdresseCase (10 o), EcrireTexte (10 o)
nombre()	× 5	50 o sur place, dans changer(), jeu() + EcrireNombre (33 o)
bouton()	× 2	24 o sur place, dans appuiStart(), jeu() + LireManette (37 o)
ecran()	× 2	21 o sur place, dans changer() + AttendreVBlank (16 o)
images()	× 2	4 o sur place, dans changer(), jeu()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

changer()	× 4	26 o sur place, dans titre(), jeu(), perdu(), main()
appuiStart()	× 2	6 o sur place, dans titre(), perdu()
titre()	× 1	3 o sur place, dans main()
jeu()	× 1	3 o sur place, dans main()
perdu()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : « LE JEU DU A ». Au titre, START lance la partie.
// Pendant 5 secondes, chaque appui sur A donne un point. Puis FINI!,
// et START ramène au titre.
//
// Ce qui est nouveau : une fonction par état, et UNE SEULE fonction, changer(),
// qui passe d'un état à l'autre. La boucle du jeu ne fait plus qu'aiguiller :
// switch (etat) appelle titre(), jeu() ou perdu().

#include <bouton> // lit un bouton de la manette
#include <ecran> // éteint ou rallume l'écran
#include <texte> // écrit un texte à l'écran
#include <nombre> // écrit un nombre en chiffres
```

2. DES CONSTANTES NOMMÉES — ETAT

```
enum Etat { TITRE, JEU, PERDU }; // Les trois écrans : TITRE = 0, JEU = 1, PERDU = 2.
```

3. UNE VARIABLE — ETAT

```
Etat etat = TITRE; // L'écran actuel.
```

4. UNE VARIABLE — SCORE

```
uint8_t score = 0; // Les points de la partie.
```

5. UNE VARIABLE — RESTE

```
uint8_t reste = 5;          // les secondes de la partie
```

6. UNE VARIABLE — TOP

```
uint8_t top = 0;           // images() au début de la seconde en cours
```

7. UNE VARIABLE — AVANTSTART

```
uint8_t avantStart = 0;    // START à l'image d'avant
```

8. UNE VARIABLE — AVANTA

```
uint8_t avantA = 0;        // A à l'image d'avant

// 1 à la première image d'un appui sur START, 0 sinon (le front).
```

9. UNE FONCTION — APPUISTART()

```
uint8_t appuiStart() {
    uint8_t s = bouton(START);          // START maintenant ?
    uint8_t front = s && !avantStart;    // maintenant ET pas avant
    avantStart = s;                      // on retient pour la prochaine fois
    return front;
}

// LA SEULE fonction qui change d'état : elle dessine le nouvel écran.
```

10. UNE FONCTION — CHANGER()

```
void changer(Etat nouvel) {
    etat = nouvel;                      // On passe au nouvel état.
    ecran(0);                           // Écran éteint pendant qu'on redessine.
    for (uint8_t l = 4; l < 14; l++) texte(0, l, "                "); // efface lignes 4 à 13

    switch (etat) {                      // Selon le nouvel état :
        case TITRE:                      // --- le titre
            texte(5, 6, "LE JEU DU A");
            texte(4, 10, "START: JOUER");
            break;                        // « break » : fin de ce cas.
        case JEU:                         // --- la partie
            score = 0;                   // en entrant dans la partie : tout repart de zéro
            reste = 5;
            top = images();              // le chrono démarre maintenant
            texte(3, 6, "APPUIE SUR A!");
            texte(3, 9, "POINTS");
            nombre(10, 9, score);
            texte(3, 10, "TEMPS");
            nombre(10, 10, reste, 1);
            break;
        case PERDU:                       // --- la fin
            texte(7, 6, "FINI!");
            texte(3, 9, "POINTS");
            nombre(10, 9, score);
            texte(4, 12, "START: TITRE");
            break;
    }
    ecran(1);                            // On rallume.
}
```

11. UNE FONCTION — TITRE()

```
void titre() {                                // Ce que fait le jeu, à chaque image, au titre :
    if (appuiStart()) changer(JEU);          //  START → la partie.
}
```

12. UNE FONCTION — JEU()

```
void jeu() {                                  // À chaque image, pendant la partie :
    uint8_t a = bouton(A);
    if (a && !avantA) {                       // un appui, un point (le front de A)
        score++;
        nombre(10, 9, score);
    }
    avantA = a;

    uint8_t ecole = images() - top;          // images depuis le début de la seconde
    if (ecole >= 60) {                       // une seconde de moins
        top = top + 60;
        reste--;
        nombre(10, 10, reste, 1);
        if (reste == 0) changer(PERDU);      // le temps est écoulé → fin
    }
}
```

13. UNE FONCTION — PERDU()

```
void perdu() {                               // À chaque image, à l'écran de fin :
    if (appuiStart()) changer(TITRE);        //  START → retour au titre.
}
```

14. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                 // Le jeu commence ici.
    changer(TITRE);                          // On dessine le titre.

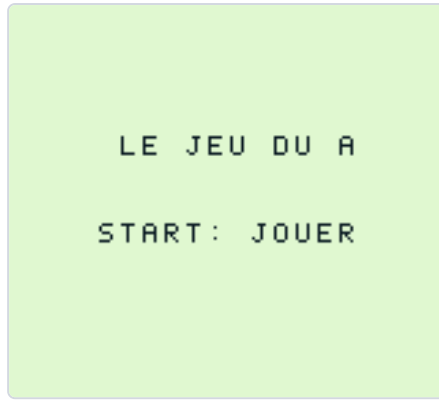
    while (true) {                           // La boucle du jeu :
        image();
        switch (etat) {                     // la boucle ne fait plus qu'aiguiller
            case TITRE: titre(); break;
            case JEU:  jeu();  break;
            case PERDU: perdu(); break;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



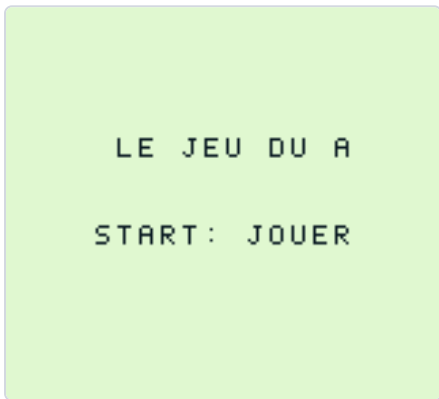
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé



START — l'écran a changé

5. Ce qu'on doit voir

LE JEU DU A ; START lance cinq secondes où chaque appui sur A marque un point ; puis FINI! et le score ; START revient au titre.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'écran titre
- ✓ START lance la partie
- ✓ sept appuis, sept points (7)
- ✓ au bout de cinq secondes, FINI! et le score (007)
- ✓ START ramène au titre

Ensuite : **119.1. La fonction sauver() — garder un nombre, même éteint** — chapitre 20 : Les états du jeu : titre, partie, fin. ·
gameboy3 — écrire une cartouche Game Boy en C++